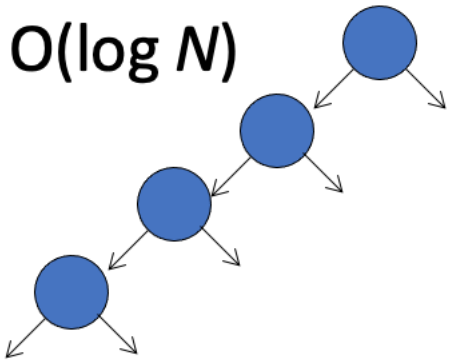
Introduction to Hashing

Problems with existing Data Structures

- *list* : store jobs in a list; remove min/max by searching (O(*N*))
  - problem: expensive to search
- *sorted list* : store in sorted list; binary search it in O(log *N*) time
  - problem: expensive to add/remove (O(*N*))
- *binary search tree* : store in BST, go right for max in O(log *N*)
  - problem: tree becomes unbalanced



HashMap

HashMap

```
HashMap<String, String> capitals = new HashMap<String, String>(50);
```

Key, value pair

Functionality is similar to ArrayList:

You can define your own key rather than using index

HashMap

A ***hash map*** is a data structure that stores unordered items by **hashing** (or mapping) each item's **key** to a location in an array.

Recall that you can use key to find corresponding value in a hash map.

E.g.

```
HashMap<String, String> capitals = new HashMap<String, String>(50);
map.put("USA", "Washington DC");
map.put("France", "Paris");
print("Capital of France is: " + map.get("France"));
```

Hashing

Hashing is the process of converting a given key into an integer.

There are many types of hash function:

In this case, we use a ***String*** hash function, because **type of key** is declared as **String**.

E.g We can *hash* ***String keys*** *into* ***ints***

…by taking the average of letter numbers in key, and round to the nearest int:

```
hash(USA)= (21+19+1)/3 = 14
hash(France) = (6+18+1+14+3+5)/6 = 8
```

Computation of index

Now, we have hash(USA) → 14, and hash(France) → 8.

Inside each **HashMap**, there is an array used to store items (let's call it **`arr []`** for now). In this case, we have a **String[] arr**, because **type of value** is declared as **String .**

Recall that arrays can only be accessed through **indices**, and indices must be **positive integers**.

In this case, we *hash* USA to 14, and France to 8, so that they can be used as a **valid index**.

Hash function & Bucket

Recall that we have created a **HashMap** object:

```
HashMap<String, String> capitals = new HashMap<String, String>(50);
```

50 is the **number of buckets**, or size of internal array (`arr[]`) used to store hash **values**.

NOT the size of HashMap!

A **hash function** computes a bucket index from item's key.

Hash function we used: Average of letter numbers in key

Bucket index we get: 14, 8

Store values in the array:

```
arr[14] = "Washington DC";
arr[8] = "Paris";
```

Hashing

Now we have a **hash map** with a bucket number (array size) of **50**, and we have stored two values inside this **hash map**.

Here's what happens when we call: `capitals.get("France");`

1. Hash function will be called to hash "France", the output will be the index.
   `hash("France")` → `8`
2. The index is 8, get value at index 8.
   `return arr[8];` `//arr[8] is "Paris".`
3. Hence, the return value of `capitals.get("France");` is "**Paris**".

String Hash Function

This time let's try something different: we can hash String by **averaging its ASCII code**.

hash("Lily") = (76+105+108+121)/4 = 103

hash("Jay") = (74+97+121)/3 = 97

Now, we have hash(Lily) → 103, and hash(Jay) → 97.

Store values in **`arr[]`**:

```
arr[103] = 87;
arr[97] = 70;
```

Get(key) method

Here's what happens when we call: `scores.get("Lily");`

1. Hash function will be called to hash "Lily", the output will be the index.
   `hash("Lily")` → `103`
2. The index is 103, get value at index 103.
   `return arr[103];` `//arr[103] is 87.`
3. Hence, the return value of `scores.get("Lily")` is **87**.

Complexity Analysis

A **HashMap**'s main advantage is that
searching (or inserting / removing) an item may require only **O(1)**
in contrast to O(N) for searching a list
or to O(log N) for binary search.

Why only O(1)?
No loops in hashing process.
Get array elements directly from memory.

Collision

Idealy, for all items that might possibly be stored in the hashmap, the hash of every key should be ***unique***, so that the hashmap's algorithms can search for a specific item by that key.

A ***collision*** occurs when an item being inserted into a hashmap maps to the *same* bucket as an existing item in the hashmap.

Collision

```
HashMap<String, Integer> scores = new HashMap<String, Integer>(130);
scores.put("Lily", 87);
scores.put("Lyli, 65");
scores.put("Jay", 70);
```

Collision!!

hash(“Lily”) = (76+105+108+121)/4 = 103

hash(“Lyli”) = (76+121+108+105)/4 = 103

Collision!!

hash(“Jay”) = (74+97+121)/3 = 97

How to resolve this?

Terminologies

Hashing

Hash function – convert any data type to non-negative integer

Data type of Key – hash function type

Data type o Value – internal array type

Buckets – internal array size

Complexity

searching (or inserting / removing) an item **may require only O(1)**

Collision

when an item being inserted into a hashmap maps to the ***same bucket*** as an existing item in the hashmap.

Collision Handling

1. **Chaining**
2. **Linear Probing**
3. **Quadratic Probing**

Chaining

```
HashMap<String, Integer> scores = new HashMap<String, Integer>(130);
scores.put("Lily", 87);
scores.put("Lyli, 65");
scores.put("Jay", 70);
```

hash(“Lily”) = (76+105+108+121)/4 = 103

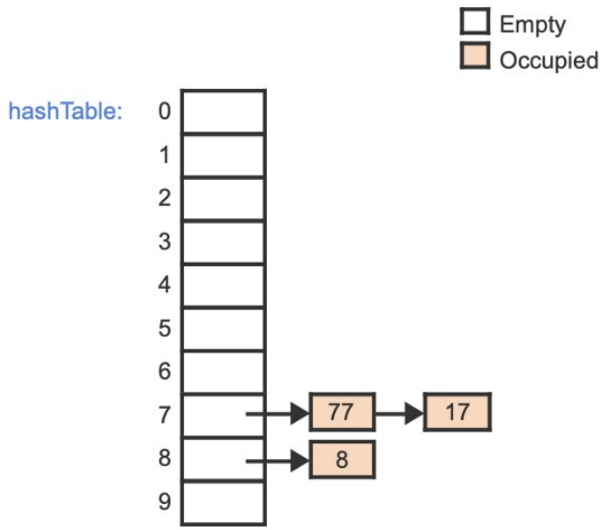
hash(“Lyli”) = (76+121+108+105)/4 = 103

hash(“Jay”) = (74+97+121)/3 = 97

Chaining (Example)

```
HashInsert(hashTable, item) {
   if (HashSearch(hashTable, item→key) == null) {
      bucketList = hashTable[Hash(item→key)]
      node = Allocate new linked list node
      node→next = null
      node→data = item
      ListAppend(bucketList, node)
   }
}

HashSearch(hashTable, key) {
   bucketList = hashTable[Hash(key)]
   itemNode = ListSearch(bucketList, key)
   if (itemNode is not null)
      return itemNode→data
   else
      return null
}
```



HashInsert(table, item 77)
HashInsert(table, item 8)
HashInsert(table, item 17)
HashSearch(table, 17) // Returns item 17

Linear Probing

A hash map with ***linear probing*** handles a collision by starting at the key's mapped bucket, and then linearly searches subsequent buckets until an empty bucket is found.

Using linear probing, a hash map *insert* algorithm uses the item's key to determine the **initial bucket**, linearly probes (or checks) each bucket, and inserts the item in the **next empty bucket**.

If the probing **reaches the last bucket**, the probing continues at bucket 0.

Returns ***true*** if the item was inserted, and returns ***false*** if all buckets are occupied.

Linear Probing (Example)

```
HashMap<String, Integer> scores = new HashMap<String, Integer>(130);
scores.put("Lily", 87);
scores.put("Lyli, 65");
scores.put("Jay", 70);
```

hash(“Lily”) = (76+105+108+121)/4 = 103

hash(“Lyli”) = (76+121+108+105)/4 = 103

hash(“Jay”) = (74+97+121)/3 = 97

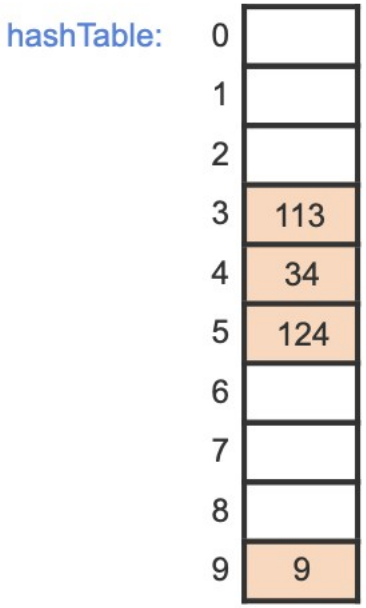
Linear Probing (Example)

HashInsert(hashTable, item 124)

HashSearch(hashTable, 124) // Returns item 124

HashSearch(hashTable, 33) // Returns null

Hash function: key % 10



Empty Bucket Types

Actually, linear probing distinguishes two types of empty buckets.

- ***empty-since-start bucket:*** bucket has been empty since the hash table was created
- ***empty-after-removal bucket:*** bucket had an item removed that caused the bucket to now be empty

The distinction will be **important during searches**, since searching only **stops for empty-since-start buckets**, not for empty-after-removal bucket.

Quadratic Probing

A hash table with ***quadratic probing*** handles a collision by starting at the key's mapped bucket, and then quadratically searches subsequent buckets until an empty bucket is found. If an item's mapped bucket is H, the formula $(H + c1 * i + c2 * i^2) \bmod (tablesize)$ is used to determine the item's index in the hash table. c1 and c2 are programmer-defined constants for quadratic probing. Inserting a key uses the formula, starting with i = 0, to repeatedly search the hash table until an empty bucket is found. Each time an empty bucket is not found, i is incremented by 1. Iterating through sequential i values to obtain the desired table index is called the ***probing sequence***.

Quadratic Probing (Example)

PARTICIPATION ACTIVITY 17.4.1: Hash table insertion using quadratic probing: c1 = 1 and c2 = 1.

1 2 3 2x speed

Hash function: key % 10
Quadratic probing sequence: (H + i + i * i) % 10

Operation	H(key)	i	Bucket index	Bucket empty?
Insert key 55	55 % 10 = 5	0	(5 + 0 + 0 * 0) % 10 = 5	Yes
Insert key 66	66 % 10 = 6	0	(6 + 0 + 0 * 0) % 10 = 6	Yes
Insert key 25	25 % 10 = 5	0	(5 + 0 + 0 * 0) % 10 = 5	No
		1	(5 + 1 + 1 * 1) % 10 = 7	Yes

hashTable:

0	
1	
2	
3	
4	
5	55
6	66
7	25
8	
9	

Empty
Occupied

i is incremented to 1 and a new index of 7 is computed. Bucket 7 is empty and 25 is inserted.

Linear Probing Example

Hash function: key % 5

HashInsert(item 49)
HashInsert(item 41)
HashInsert(item 42)
HashInsert(item 44)

HashInsert(item 46)

HashRemove(item 49)

HashSearch(44)

Hash Functions

A good hash function minimizes collisions!

A ***perfect hash function*** maps items to buckets with **no collisions**.

A ***perfect hash function*** can be created if the number of items and all possible item keys are known beforehand. The runtime for insert, search, and remove is ***O(1)*** with a perfect hash function.

With ***chaining***, a good hash function results in **short bucket lists** and thus fast inserts, searches, and removes.

With ***linear probing***, a good hash function will **avoid hashing multiple items to consecutive buckets** and thus minimize the average linear probing length to achieve fast inserts, searches, and removes.

On average, a good hash function will achieve **O(1)** inserts, searches, and removes, but in the worst-case may require **O(N)**.

Hash Map resize

Resize operation increases the number of buckets, while preserving all existing items. A hash table with N buckets is commonly resized to the **next prime number ≥ N * 2**.

A new array is allocated, and all items from the old array are re-inserted into the new array, making the resize operation's time complexity ***O(N)***.